

EXPERIMENT – 01

BANK ACCOUNT MANAGEMENT SYSTEM

Experiment No.: 01

Date: _____

Title: Bank Account Management System using Classes and Objects

■ Objective / Aim

To design and implement a Bank Account Management System in Java using classes and objects. The system manages different types of bank accounts and performs operations such as deposit, withdrawal, interest calculation, transfer, and display of account details.

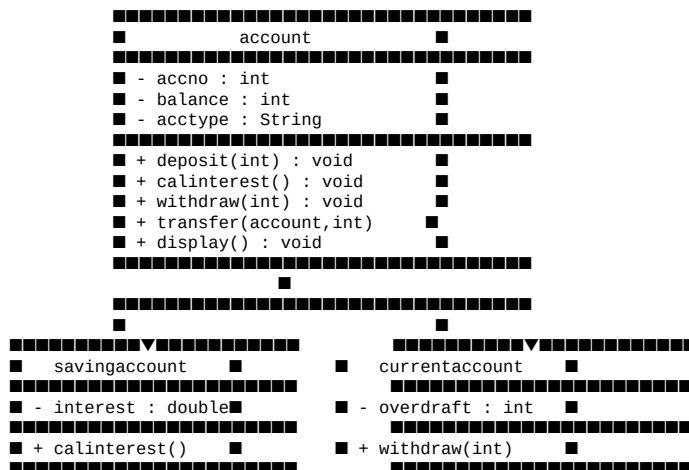
■■ System Requirements

Hardware	Standard PC/Laptop; Minimum 4 GB RAM; sufficient storage; keyboard and monitor.
Software	Windows / Linux / macOS; Java Development Kit (JDK 17 or later); any Java IDE or text editor; Command Prompt / Terminal

■■ Algorithm / Logic Description

1. Create the base class account with account number, balance, and account type.
2. Define a constructor to initialize the account details.
3. Define deposit() to add the given amount to the account balance.
4. Define withdraw() to withdraw an amount when sufficient balance is available.
5. Define calinterest() as a method for interest calculation.
6. Define transfer() to transfer money from one account to another account.
7. Define display() to display account number, account type, and balance.
8. Create savingaccount by extending account and add an interest attribute.
9. Override calinterest() in savingaccount to calculate interest based on balance.
10. Create currentaccount by extending account and add an overdraft limit.
11. Override withdraw() in currentaccount to allow withdrawal within the overdraft limit.
12. Create objects s1 and c1 in main() and perform deposit, withdrawal, interest calculation, and transfer operations.
13. Display the final account details.

■ UML Class Diagram



savingaccount and currentaccount inherit from account.

■ Source Code

The following is the corrected version of the code supplied for the experiment. The original class names and overall structure are retained; only the required Java errors were corrected so that the program compiles and demonstrates inheritance and method overriding.

```
class account {
    int accno;
    int balance;
    String acctype;

    public account(int accno, int balance, String acctype) {
        this.accno = accno;
        this.balance = balance;
        this.acctype = acctype;
    }

    void deposit(int amount) {
        balance = balance + amount;
        System.out.println("balance after deposit is:" + balance);
    }

    void calinterest() {
    }

    void withdraw(int amount) {
        if (balance >= amount) {
            balance = balance - amount;
            System.out.println("balance after withdrawal is :" + balance);
        } else {
            System.out.println("insufficient balance");
        }
    }

    void transfer(account recevier, int amount) {
        if (balance >= amount) {
            balance = balance - amount;
            recevier.balance = recevier.balance + amount;

            System.out.println("balance after transfer is :" + recevier.balance);
        } else {
            System.out.println("insufficient balance");
        }
    }

    void display() {
        System.out.println(accno);
        System.out.println(acctype);
        System.out.println(balance);
    }
}

class savingaccount extends account {
    double interest;

    public savingaccount(int accno, int balance, double interest) {
        super(accno, balance, "saving");
        this.interest = interest;
    }

    @Override
    void calinterest() {
        double interestAmount = interest * balance / 100;
        System.out.println("interest is:" + interestAmount);
    }
}

class currentaccount extends account {
    int overdraft;

    public currentaccount(int accno, int balance, int overdraft) {
        super(accno, balance, "current account");
        this.overdraft = overdraft;
    }

    @Override
    void withdraw(int amount) {
        if (amount <= balance + overdraft) {
            balance = balance - amount;
            System.out.println("Amount Withdrawn: " + amount);
        } else {
            System.out.println("Overdraft Limit Exceeded");
        }
    }
}

public class BankManagement {
    public static void main(String[] args) {

        savingaccount s1 =
            new savingaccount(101, 20000, 5);
        currentaccount c1 =
```

```

        new currentaccount(102, 15000, 5000);

System.out.println("Savings Account");
s1.display();

s1.deposit(5000);
s1.withdraw(3000);
s1.calinterest();

System.out.println("\nCurrent Account");
c1.display();

c1.deposit(2000);
c1.withdraw(18000);

System.out.println("\nTransfer Money");
s1.transfer(c1, 4000);

System.out.println("\nFinal Details");

s1.display();
c1.display();
    }
}

```

■ Compilation & Execution Commands

Save the source code as:

BankManagement.java

Compile:

javac BankManagement.java

Run:

java BankManagement

■ ■ Output Screen

```
Savings Account
101
saving
20000
balance after deposit is:25000
balance after withdrawal is :22000
interest is:1100.0
```

```
Current Account
102
current account
15000
balance after deposit is:17000
Amount Withdrawn: 18000
```

```
Transfer Money
balance after transfer is :18000
```

```
Final Details
101
saving
18000
102
current account
18000
```